TeleCon Control Protocol

Byte-stream contract with the Android application

Firmware reference — any MCU that implements this contract

Version 1.2 2026

Abstract

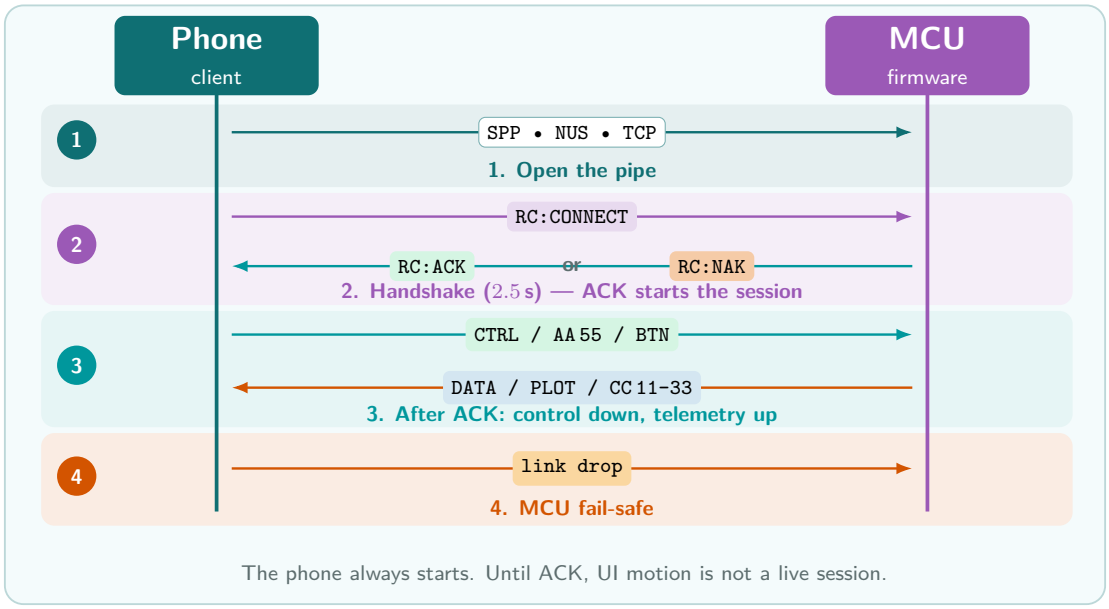
This document specifies how firmware must exchange bytes with TeleCon4ESP32 over Classic Bluetooth SPP, BLE Nordic UART, or TCP. The application is transport-agnostic: it does not require an Espressif SoC, a particular GAP name, or vendor AT commands. Official Arduino sketches are reference implementations of this contract, not part of the wire definition.

Control Panel and RC Vehicle Pro share the application prefix `RC`. HTTP camera streaming (`/stream`, `/capture`) is a separate session and is out of scope.

1 Architecture

The phone is always the client. After the underlying link is established, it runs an application-layer session. Three layers must be implemented on the device:

1. **Transport** — a reliable-enough bidirectional octet stream (Section 2).
2. **Session handshake** — one text `CONNECT` line and an `ACK`/`NAK` reply (Section 4). Timeout on the phone is 2.5 s. Until ACK, UI control is not a live TeleCon session even if the OS reports the socket as connected.
3. **Payload** — Simple text lines (Section 7) or Binary frames (Section 8), selected by the user in application settings and echoed in `CONNECT`'s `proto` field.



1 pipe | 2 CONNECT | ACK / CTRL | NAK | DATA / PLOT | 4 drop

Phone–MCU flow. Green = accept, orange = reject, teal = control, blue = telemetry.

The Bluetooth advertised name is GAP/SDP presentation only (for example `ESP32-TC-RC-BT-Simple` on official firmware). The handshake parser never matches the substring `ESP32`. Connect uses the remote **address** plus SPP UUID or NUS attributes.

1.1 Mode coupling

The firmware's transport and payload must match the stored `BluetoothConnectionMode`:

Application mode	Transport	proto	Payload after ACK
Classic Simple	SPP	`simple`	Section 7
Classic Binary	SPP	`binary`	Section 8
BLE Binary	NUS	`binary`	Section 8

Simple | Binary | Legacy Simple

BLE is Binary-only in the current application. A custom GATT serial profile (typical HM-10 UUIDs) will fail service discovery: the client looks up Nordic UART only.

Mismatch should be signalled with `NAK` (`expected` = device capability, `actual` = value from `CONNECT`). Silent ignore produces `HandshakeFailure.Timeout` on the phone.

2 Transports

2.1 Classic SPP (RFCOMM)

Serial Port Profile, UUID `00001101-0000-1000-8000-00805F9B34FB`.

The Android client attempts, in order: secure `createRfcommSocketToServiceRecord`, insecure `createInsecureRfcommSocketToServiceRecord`, then a reflective RFCOMM socket on channel 1. The third attempt exists because some stacks (including ESP32 `BluetoothSerial` while a SoftAP `WifiNetworkSpecifier` is held) fail SDP UUID lookup.

HC-05 / HC-06 expose this UUID as a transparent UART bridge. The MCU only implements the byte protocol on that UART; baud is a module setting (commonly 9600 or 115200), not negotiated by the phone.

2.2 BLE Nordic UART Service

Attribute on the peripheral	UUID	Role
Primary service	`6E400001-B5A3-F393-E0A9-E50E24DCCA9E`	Service
RX (Write / Write Without Response)	`6E400002-...CCA9E`	Phone → device
TX (Notify)	`6E400003-...CCA9E`	Device → phone
CCCD on TX	`00002902-0000-1000-8000-00805F9B34FB`	Enable notify

Service | Phone writes | Device notifies | Client config

The client writes protocol octets to RX and reassembles TX notifications into the same stream used for SPP. Each notify/write must be $\leq$ MTU-3. Default ATT MTU 23 implies 20-byte chunks; request a larger MTU if you send Binary frames in one shot. Handshake text is still newline-delimited ASCII on this stream.

2.3 TCP

Same session and payload as Classic, on a single TCP connection. Official SoftAP firmware binds `192.168.4.1:3333`. Host/port are not part of the application protocol; they are deployment parameters. Do not invent a second handshake for TCP.

3 Text encoding

Handshake and Simple messages. Colour marks each token's job:

APP	TYPE	key1	value1	key2	value2
app	type	key	value	key	value

```
APP:TYPE,key1,value1,key2,value2,...
```

App Type Key Value

Terminated by `LF` (`0x0A`). A preceding `CR` is stripped. `APP` for this product is `RC`. Keys are case-sensitive as sent by the phone (`proto`, `lx`, ...).

Boolean values: `1/0`, `true/false`, `on/off`, or any non-zero integer.

The inbound assembler treats a leading `0xCC` as the start of a Binary telemetry frame and printable ASCII as text. After ACK in Binary mode, `0xAA`/`0xBB` are phone-to-device control frames (they do not go through the text line parser on the device unless you implement that yourself).

4 Session handshake

Required on every new SPP, NUS, or TCP connection. Not used for HTTP camera.

4.1 Client → device

```
RC:CONNECT,proto,simple
RC:CONNECT,proto,binary
RC:CONNECT,proto,wifi
```

App Type Key Value

`wifi` is retained for legacy one-board CAM SoftAP. New Simple firmware should accept both `simple` and `wifi` and emit the same ACK.

The phone waits 2500 ms for a line with the same app prefix `RC` and type `ACK` or `NAK`.

4.2 Device → client

```
RC:ACK,app,RC
```

Additional ACK keys are ignored for session establishment.

```
RC:NAK,reason,proto_mismatch,expected,simple,actual,binary
RC:NAK,reason,app_mismatch,expected,RC,actual,XX
```

`reason` must be `proto_mismatch` or `app_mismatch` for the structured Android dialogs. Only `RC:ACK,app,RC` establishes the session. A later ACK is not CONNECT success. Stock sketches do not send a steer-center ACK.

4.3 Recommended device state

1. Link up → `session_ok = false`, fail-safe outputs.
2. Parse complete `CONNECT` line → compare `proto` (and optionally app prefix) → ACK or NAK.
3. On ACK, enable `CTRL/AA 55` handling and telemetry TX.
4. On link drop, reset `session_ok` and outputs.

5 Reference implementation: SPP Simple (ESP32 `BluetoothSerial`)

Classic Bluetooth on ESP32-WROOM-class parts. Application setting: Classic Simple. The GAP name is arbitrary.

```
#include "BluetoothSerial.h"

#define SERIAL_BAUD         115200

BluetoothSerial SerialBT;

const char* BT_NAME = "TeleCon-RC";
const char* MY_PROTO = "simple";

bool sessionOk = false;
String lineBuf;

int lx = 0, ly = 0, rx = 0, ry = 0;
int lk = 512, rk = 512;
uint8_t sw = 0;

String valueOf(const String& line, const char* key) {
  String token = String(key) + ",";
  int i = line.indexOf(token);
  if (i < 0) return "";
  i += token.length();
  int j = line.indexOf(',', i);
  if (j < 0) j = line.length();
  return line.substring(i, j);
}

void sendLine(const String& s) {
  SerialBT.print(s);
  SerialBT.print('\n');
}

void applyOutputs() {
  // Map already-conditioned percent sticks (-100..100) to actuators.
}

void handleConnect(const String& line) {
  String proto = valueOf(line, "proto");
  if (proto != MY_PROTO &&
      !(String(MY_PROTO) == "simple" && proto == "wifi")) {
    sendLine(String("RC:NAK,reason,proto_mismatch,expected,") +
             MY_PROTO + ",actual," + proto);
    sessionOk = false;
    return;
```

```
  }
  sendLine("RC:ACK,app,RC");
  sessionOk = true;
}

void handleCtrl(const String& line) {
  lx = valueOf(line, "lx").toInt();
  ly = valueOf(line, "ly").toInt();
  rx = valueOf(line, "rx").toInt();
  ry = valueOf(line, "ry").toInt();
  lk = valueOf(line, "lk").toInt();
  rk = valueOf(line, "rk").toInt();
  sw = (uint8_t)strtol(valueOf(line, "sw").c_str(), nullptr, 16);
  applyOutputs();
}

void handleLine(String line) {
  line.trim();
  if (line.length() == 0) return;
  if (line.startsWith("RC:CONNECT")) {
    handleConnect(line);
    return;
  }
  if (!sessionOk) return;
  if (line.startsWith("RC:CTRL")) handleCtrl(line);
  else if (line.startsWith("RC:BTN")) {
    (void)valueOf(line, "id").toInt();
  } else if (line.startsWith("RC:SET")) {
    // Stock sketches ignore RC:SET. Trim stays on the phone.
  }
}

void setup() {
  Serial.begin(SERIAL_BAUD);
  SerialBT.begin(BT_NAME);
}

void loop() {
  if (!SerialBT.hasClient()) {
    if (sessionOk) {
      sessionOk = false;
      // fail-safe
    }
    lineBuf = "";
  }
  while (SerialBT.available()) {
    char c = (char)SerialBT.read();
    if (c == '\n') {
      handleLine(lineBuf);
```

```
      lineBuf = "";
    } else if (c != '\r') {
      if (lineBuf.length() < 200) lineBuf += c;
      else lineBuf = "";
    }
  }
}
```

Telemetry after ACK (typical 2 Hz DATA, 10–20 Hz PLOT):

```
void sendTelemetry(int leftPanel, int batt, int analog) {
  sendLine("RC:DATA,left," + String(leftPanel) +
           ",batt," + String(batt) + ",analog," + String(analog));
}

void sendPlot(uint8_t v0, uint8_t v1, uint8_t v2, uint8_t v3,
              uint8_t v4, uint8_t v5, uint8_t v6, uint8_t v7) {
  sendLine("RC:PLOT,v0," + String(v0) + ",v1," + String(v1) +
           ",v2," + String(v2) + ",v3," + String(v3) +
           ",v4," + String(v4) + ",v5," + String(v5) +
           ",v6," + String(v6) + ",v7," + String(v7));
}
```

6 Reference implementation: SPP Simple via HC-05 UART

The module terminates SPP; the MCU sees a UART. Same `CONNECT/CTRL` grammar. Match module baud (often 9600). Cross TX/RX; level-shift if required.

```
#include <Arduino.h>

#define SERIAL_BAUD         115200

HardwareSerial BT(1);
const int BT_RX_PIN = 16;
const int BT_TX_PIN = 17;
const uint32_t BT_BAUD = 9600;
const char* MY_PROTO = "simple";

bool sessionOk = false;
String lineBuf;

String valueOf(const String& line, const char* key) {
  String token = String(key) + ",";
  int i = line.indexOf(token);
  if (i < 0) return "";
  i += token.length();
  int j = line.indexOf(',', i);
  if (j < 0) j = line.length();
```

```
  return line.substring(i, j);
}

void sendLine(const String& s) {
  BT.print(s);
  BT.print('\n');
}

void handleConnect(const String& line) {
  String proto = valueOf(line, "proto");
  if (proto != MY_PROTO) {
    sendLine(String("RC:NAK,reason,proto_mismatch,expected,") +
             MY_PROTO + ",actual," + proto);
    sessionOk = false;
    return;
  }
  sendLine("RC:ACK,app,RC");
  sessionOk = true;
}

void handleLine(String line) {
  line.trim();
  if (line.startsWith("RC:CONNECT")) { handleConnect(line); return; }
  if (!sessionOk) return;
  if (line.startsWith("RC:CTRL")) {
    int ly = valueOf(line, "ly").toInt();
    int rxStick = valueOf(line, "rx").toInt();
    (void)ly;
    (void)rxStick;
  }
}

void setup() {
  Serial.begin(SERIAL_BAUD);
  BT.begin(BT_BAUD, SERIAL_8N1, BT_RX_PIN, BT_TX_PIN);
}

void loop() {
  while (BT.available()) {
    char c = (char)BT.read();
    if (c == '\n') { handleLine(lineBuf); lineBuf = ""; }
    else if (c != '\r') {
      if (lineBuf.length() < 200) lineBuf += c;
      else lineBuf = "";
    }
  }
}
```

7 Simple payload

7.1 Phone → device

RC:CTRL Emitted on UI change (not a fixed phone-side poll).

```
RC:CTRL,lx,-50,ly,100,rx,0,ry,0,lk,512,rk,256,sw,03
```

Key	Domain	Meaning
lx, ly, rx, ry	−100 . . . 100	Sticks
lk, rk	0 . . . 1023	Knobs
sw	8-bit hex, two digits	S1 = bit0 . . . S8 = bit7

Sticks Knobs Switches

Dual-rate, expo, reverse, deadzone, and steering trim are applied on the phone before TX. Firmware must not apply the same curves again. Stock sketches do not store a servo center: stick rx = 0 is a 1500 µs pulse. Phone trim (rightTrimX) is already inside the rx value. A later vehicle sketch may store one center by itself if a second transmitter must share that straight-ahead.

RC:BTN RC:BTN,id,1 — ids 1–4, the center cluster. The phone does not send id 16.

RC:SET Stock sketches ignore RC:SET. Do not persist a steer center from this line.

7.2 Device → phone

Telemetry is optional. Send it only after ACK, on the same text stream as CTRL. Two line types cover the HUD: DATA for slow gauges and panels, PLOT for a fast analog bus. Omit either type if the firmware has nothing to show; the phone keeps the last values.

RC:DATA — panels, gauges, LED (typically 2 Hz) One line is a snapshot of integer HUD fields. Keys may appear in any order. Missing keys leave the previous value on the phone.

```
RC:DATA,left,1234,right,5678,analog,42,batt,88,led,0F
```

Key	Domain	Meaning	Typical HUD
left	integer	Left numeric panel	RC Vehicle Pro: speed×10
right	integer	Right numeric panel	Second counter / unused
analog	0 . . . 255	Analog gauge	Motor temperature
batt	0 . . . 255	Battery gauge	Charge 0 . . . 100 is enough
led	8-bit hex, two digits	LED bitfield	Status lamps

Panels Gauges LEDs

Panel on/off and colour are Android settings, not wire fields. Legacy keys `lo`, `ro`, `lg`, `rg`, `lt`, `rt`, `at`, `bt` are ignored for appearance.

The phone may remap these keys (and PLOT channels) onto plots, radar, or gauges in RC settings. Firmware must keep sending the same keys. Do not emit `RC:RADAR` or channel-map packets.

`RC:PLOT` — analog samples (typically 10–30 Hz) Each line is one time sample on a numbered bus `v0`…`v7` (= CH1…CH8). Map sensors to 0…255 (0 = bottom of the graph, 255 = top). Stock Bluetooth sketches send all eight keys about every 50 ms. A four-sample line is still valid; omitted `v4`…`v7` stay at 0 on the phone. The app keeps eight channels and the last 100 points on each; a ninth sample is dropped.

```
RC:PLOT,v0,128,v1,200,v2,64,v3,180
RC:PLOT,v0,128,v1,200,v2,64,v3,180,v4,0,v5,0,v6,0,v7,0
```

Key	Channel	Range	Notes
v0	CH1	0…255	Default first plot trace
v1	CH2	0…255	Default second trace / radar
v2	CH3	0…255	Default third trace
v3	CH4	0…255	Default fourth trace
v4…v7	CH5…CH8	0…255	Same bus; bind in the channel map

CH1–CH2 CH3–CH4 CH5–CH8

Control Panel shows four plot widgets (defaults CH1–CH4). Trace colours are fixed by channel index. Plot labels and the channel map live in Android RC settings — do not send `RC:PLOTCFG`. The same eight bytes also feed radar, gauges, and HUD badges when the user selects that channel.

Stock Bluetooth channel table. Classic Simple, Classic Binary, and BLE Binary fill the eight bytes from `kTelemetryChannels[]` (row 0 = CH1). Classic Simple keeps the table in the `.ino`; the binary sketches keep it in `TelemetryChannels.h`. A row is `{ kind, vehicle, pin, ledcChannel, dutyMin, dutyMax }`. Only the fields for `kind` are read.

kind	Byte	Fields used
TEL_VEHICLE	Built-in signal named by `vehicle`	`vehicle`
TEL_ADC	`analogRead`, 0…4095 ↦ 0…255	`pin`
TEL_GPIO	`digitalRead` ↦ 0 or 255	`pin`
TEL_PWM	LEDC duty, `dutyMin`↦ 0, `dutyMax`↦ 255	`pin`, channel, duty window

On a pin row the `vehicle` name is ignored. On a `TEL_VEHICLE` row the pin and duty window are ignored. Built-in names are temperature, throttle (ly), battery, steer (rx), lights, both knobs, and ry. Those magnitudes do not show direction. `TEL_GPIO` on an

output pin reports the level last written. `TEL_PWM` reads a pin that is already attached. An 8-bit motor uses duty 0 . . . 255. A 12-bit servo across 1000–2000 µs uses duty 204 . . . 409 (center ≈ 128). Core 3.x reads the GPIO; core 2.x reads `ledcChannel`.

Classic Simple defaults: demo temperature, throttle magnitude, demo battery, steering-stick magnitude, lights switch, both knobs, then the steering-servo PWM. Classic Binary and BLE Binary defaults, with vehicle IO on: temperature, throttle magnitude, battery scaled to 0 . . . 255, steering-stick magnitude, left-drive PWM (GPIO 25), right-drive PWM (GPIO 27), lights GPIO 15, pan-servo PWM (GPIO 12, same 204 . . . 409 window).

Both axes of both sticks fit in four of the eight rows. Map a signed axis with center 128: `map(axis, -100, 100, 0, 255)`. The plot period stays 50 ms; filling more of the eight bytes does not add a packet. One Simple line is about 80 bytes. Binary `CC 33` with eight samples is 14 bytes.

Typical `loop()`: PLOT every 30–100 ms from ADC or filtered sensors; DATA every 500 ms for panels and battery. Gate both on `session_ok`.

Drop this into the sketch from Section 5. `sendLine`, `sessionOk`, `ly`, `rx`, `lk`, `rk`, and `sw` are already there.

```
int toPlotByte(int value, int minVal, int maxVal) {
  if (maxVal <= minVal) return 0;
  return constrain(map(value, minVal, maxVal, 0, 255), 0, 255);
}

void sendData(int left, int right, int analog, int batt, uint8_t led) {
  char line[96];
  snprintf(line, sizeof(line),
           "RC:DATA,left,%d,right,%d,analog,%d,batt,%d,led,%02X",
           left, right, analog, batt, led);
  sendLine(line);
}

void sendPlot(uint8_t v0, uint8_t v1, uint8_t v2, uint8_t v3,
              uint8_t v4, uint8_t v5, uint8_t v6, uint8_t v7) {
  char line[128];
  snprintf(line, sizeof(line),
           "RC:PLOT,v0,%u,v1,%u,v2,%u,v3,%u,v4,%u,v5,%u,v6,%u,v7,%u",
           v0, v1, v2, v3, v4, v5, v6, v7);
  sendLine(line);
}

unsigned long lastDataMs = 0;
unsigned long lastPlotMs = 0;

void loop() {
  // read SerialBT and handleLine() as in Section 5
  if (!sessionOk) return;

  unsigned long now = millis();
  if (now - lastDataMs >= 500) {
    lastDataMs = now;
```

```
    int speedX10 = abs(ly) * 25 / 10;  // RC Vehicle Pro HUD
    sendData(speedX10, 0, 90, 76, sw);
  }
  if (now - lastPlotMs >= 50) {
    lastPlotMs = now;
    sendPlot(toPlotByte(analogRead(34), 0, 4095),
             toPlotByte(abs(ly), 0, 100),
             toPlotByte(76, 0, 100),
             toPlotByte(abs(rx), 0, 100),
             (sw & 0x01) ? 255 : 0,
             toPlotByte(lk, 0, 1023),
             toPlotByte(rk, 0, 1023),
             128);
  }
}
```

8 Binary payload

Used after `RC:CONNECT,proto,binary` and ACK. Handshake remains ASCII.

8.1 Phone → device — `AA 55` (18 bytes)

Little-endian 12-bit channels. Colours mark the job of each field:

AA 55	Lx	Ly	Rx	Ry	Lk	Rk	SW	∑	CR LF
0–1	2–3	4–5	6–7	8–9	10–11	12–13	14	15	16–17

Offset	Size	Content	Role
0–1	2	`0xAA 0x55` frame sync	Sync
2–3	2	Lx, 12-bit LE	Channel
4–5	2	Ly, 12-bit LE	Channel
6–7	2	Rx, 12-bit LE	Channel
8–9	2	Ry, 12-bit LE	Channel
10–11	2	Left knob 0 . . . 1023, 12-bit LE	Knob
12–13	2	Right knob 0 . . . 1023, 12-bit LE	Knob
14	1	Switch bitfield (S1 = bit0 . . . S8 = bit7)	Switches
15	1	∑ bytes 2 . . . 14 (low 8 bits)	Checksum
16–17	2	`0x0D 0x0A` trailer	Terminator

Sync Channels Knobs Switches Checksum Terminator

Stick mapping: percent $p \in [-100, 100] \mapsto \lfloor (p+100) \times 4094/200 \rfloor$ with center $0 \mapsto 2047$ (unsigned 0 . . . 4094).

8.2 Phone → device — `BB 66` (4 bytes)

Ids match Simple `RC:BTN`.

Offset	Size	Content	Role
0–1	2	`0xBB 0x66` frame sync	Sync
2	1	Button id (1–4, center cluster)	Button
3	1	Same id repeated	Button

Sync Button

8.3 Device → phone

`CC 11` (8 bytes)

Offset	Size	Content	Role
0–1	2	`0xCC 0x11` frame sync	Sync
2–3	2	left, int16 LE	Telemetry
4–5	2	right, int16 LE	Telemetry
6	1	Reserved flags	Telemetry
7	1	$\sum$ bytes 2 . . . 6 (low 8 bits)	Checksum

`CC 22` (6 bytes)

Offset	Size	Content	Role
0–1	2	`0xCC 0x22` frame sync	Sync
2	1	analog, $u8$	Telemetry
3	1	battery, $u8$	Telemetry
4	1	LED, $u8$	Telemetry
5	1	$\sum$ bytes 2 . . . 4 (low 8 bits)	Checksum

`CC 33` (variable) Stock Bluetooth sketches send `count = 8` (14-byte frame). `count = 4` remains valid; the phone then leaves CH5–CH8 at 0. Do not send `CC 44`. The eight samples are the same CH1–CH8 bus as Simple `RC:PLOT`, filled from `kTelemetryChannels[]` in `TelemetryChannels.h`.

Offset	Size	Content	Role
0–1	2	`0xCC 0x33` frame sync	Sync
2–3	2	`length` uint16 LE ($= 1 +$ *count*)	Length
4	1	`count` (4 or 8)	Count
5–4+c	c	c samples $u8$ (CH1. . . CHc)	Samples
5+c	1	$\sum$ from offset 2 to last sample	Checksum

$c =$ `count`. Checksum covers offset 2 up to the last sample.

9 Out of scope

- HTTP MJPEG and `/camconfig`.
- Official SoftAP SSID assignment (deployment, not framing).
- Non-NUS BLE serial profiles.
- Optional handshake; the client will not confirm the session.

10 Implementation checklist

1. Bind SPP, NUS, or TCP as an octet stream.
2. Buffer until `LF`; parse `RC:CONNECT`.
3. Reply `RC:ACK,app,RC` or a structured `NAK`.
4. Gate control/telemetry on `session_ok`; fail-safe on disconnect.
5. Simple: `CTRL`/`BTN`/`SET` and optional `DATA`/`PLOT`.
6. Binary: `AA 55`/`BB 66` and optional `CC 11`/`CC 22`/`CC 33`.
7. Document which application mode the binary was built for.